

Date Submitted: April 26th, 2026

GROUP 22 L01 - Winter 2026

ENEL 300 - *Electrical and Computer Engineering Professional Skills*

RC CAR PROJECT FINAL DESIGN REPORT

Anaya Adil 30240593

Daanya Ahmed 30263025

Lama Alkhatib 30141065

Lamia Waseem 30243122

Schulich School of Engineering, *Department of Electrical and Software Engineering*
University of Calgary

Table of Contents:

Abstract	3
Background	3
1.Electrical Architecture	4
1.1.Power Board Electrical Architecture (Power Board PCB)	4
1.1.0.Voltage Regulation and Power Loss	
1.1.1.MOSFET Thermal Management	
1.1.2.Stabilization Capacitors	
1.1.3.Resistors and Signal Conditioning	
1.1.4.Noise and Grounding	
1.1.5.Motor Driver Circuit Design	
1.2.Remote Control Electrical Architecture (RC PCB)	9
1.2.1.System Integration and Signal Flow	
1.2.2.PCB Layout and Routing Strategy	
1.2.3.Signal Integrity and Voltage Interface Challenges	
1.2.4.Decoupling and Passive Component Usage	
2.Metal Detection	12
2.1.Phase 1: NE555 Timer Circuit	
2.2.Phase 2: Transistor-Based Oscillator Circuit	
2.3.Phase 3: Multi-Stage Induction Balance	
2.4.Phase 4: Digital Pulse Induction	
2.5.Phase 5: Contact-Based Metal Detection	
3.Distance Sensing	15
4.PCB Routing Strategy and Layout Considerations	16
4.1.Power PCB Routing (High-Current Design)	
4.2.Control PCB Routing (Signal Integrity Design)	
4.3.Grounding and Return Path Design	
4.4.Decoupling and Placement Strategy	
4.5.Routing Challenges and Practical Modifications	
4.6.Design Iteration Without Full Redesign	
5.Remote Controller System Design	20
5.1.Inputs and Outputs	
5.2.Remote Control Code Design	
6.Code Design	20
6.1.Car Code Design	
7.Software Architecture	21
8.Software Design	22
9.Mechanical Architecture	23
9.1.Mechanical Design	
9.2.Drivetrain Design and Rationale	
9.3.Chassis Design and Rationale	
9.4.Shelves and Mounting Strategy	
9.5.Materials Selection	
9.6.Mechanical Constraints and Challenges	
9.7.Testing and Validation	
9.8.Limitations and Future Improvements	
10.References	29
Appendices	32

Abstract

This report investigates the design and development of an All-Wheel-Drive (AWD) remote-controlled vehicle integrated with real-time metal detection and ultrasonic object-sensing systems. The primary objective was to create a system capable of navigating a twenty-degree incline while accurately identifying copper plates and measuring distances to certain obstacles. The project followed an iterative design process, where every subsystem, from the mechanical chassis to the metal detection circuitry, was refined across multiple stages to optimize final performance.

The final design utilizes an ESP32 microcontroller for Bluetooth communication and motor control, paired with a 3D printed chassis optimized for component organization. The vehicle's power management was strategically designed; a 12V rechargeable battery powering the drivetrain, a 9V battery dedicated for stable operation of the metal detection system, and a separate 9V battery powering the remote control. Results from the final testing demonstrate high accuracy in object detection and good mechanical durability. By overcoming significant engineering challenges throughout the development process, we successfully produced a functional, high-performance RC vehicle.

Background

The design was based on several sources such as tutorials, datasheets and documentation that implemented the fundamental concepts of programming to the practical execution of hardware within an embedded system with several peripherals. Overall, the references enabled communication between components and the embedded system while successfully producing a final design.

The LCD module was developed using the NHD-0116AZ-FL-YBW datasheet [2], which provided detailed information on the structure and pin configurations. With the additional assistance from resource [1], modifications were made regarding the grounding of the read/write pin(RW) so the LCD would operate solely in write mode. This simplification reduced the complexity of the program while removing the need for any read operations associated with the interface. The example was slightly outdated due to the updates the ESP-IDF had undergone so functions like `gpio_pad_select_gpio()` were replaced with updated equivalents like `esp_rom_gpio_pad_select_gpio()`.

The ultrasonic system is constructed using a HC-SR04 sensor, along with the assistance from ESP-IDF based document and tutorial [3]–[5]. These references aided in the formatting of the trigger/echo pulses, conversion from time measurements to distance and eventually the installment of timing accuracy in delays and loops.

The wireless communication between the car and controller was accomplished using ESP-NOW, a protocol that allows esp devices to communicate by using an initialization process along with a peer-to-peer model that implements callback functions. The references were fundamental in understanding the general process and narrowing down the exact steps required for the process.

To develop the joystick code two varying sources were used, a high-level C++ abstraction program [14] and a lower-level example program. The former had several compatibility issues due to the C++ library so an ADC based program was established instead. As such, official ESP-IDF documents were used to configure and read analog inputs and make up for the rather outdated examples used, primarily due to the

several updates in the IDE. To develop a further understanding on how ADC worked, its functions and updates references [15] and [16] were used.

The motor driver code was essentially based on the ESP IDF PWM documentation[11] and prior experience with arduino based implementation with a L298N motor driver[12]. Similarly, the button logic was adapted from prior coursework[10], including pull-up resistor configuration and level triggered logic to ensure accurate readings.

1. Electrical Architecture

1.1. Power Board Electrical Architecture(Power Board PCB)

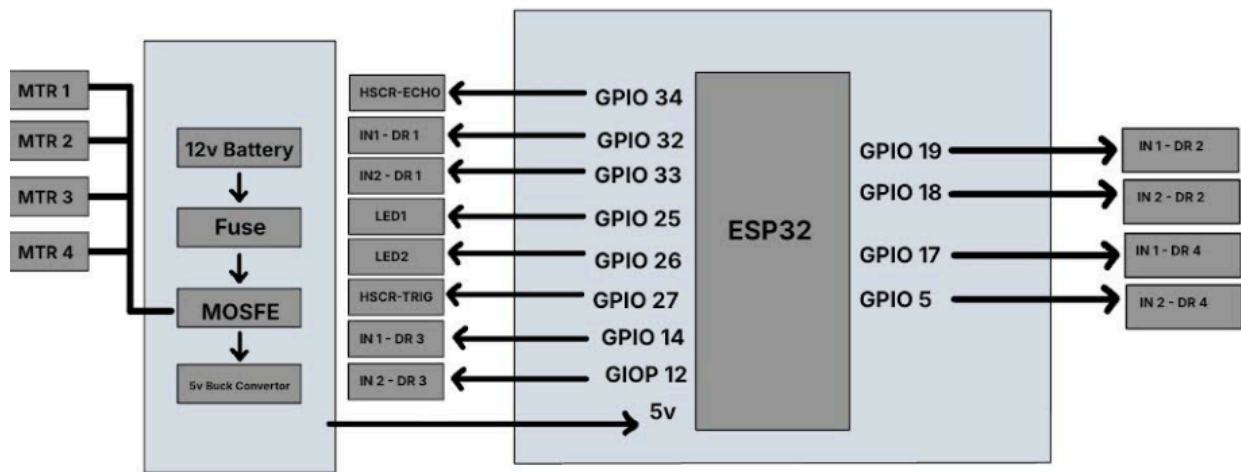


Fig.1. Power Board Electrical Architecture

The power board distributes energy from the 12 V battery through a protection chain consisting of a fuse and P-channel MOSFET, then separates the system into a high-current motor path and a regulated logic path. The protected rail labeled 12V_SAFE / 12V_FUSED supplies the motor driver section, while the buck converter generates the 5 V rail used by the ESP32, HC-SR04 sensor, LEDs, and other low-power circuitry. The IRF4905 is suitable for this protection path because it is a P-channel MOSFET with high current capability and low on-resistance when properly driven.[19]

1.1.0. Voltage Regulation and Power Loss

A major design concern is reducing heat in the voltage regulation stage. If a linear 7805 regulator were used from 12 V to 5 V, the power loss would be:

$$P_{\text{loss}} = (V_{\text{in}} - V_{\text{out}})I$$

For example, if the logic rail draws 1A:

$$(12V-5V).(1A)=7W$$

This is too high for a small regulator without serious heat sinking. With a typical TO-220 thermal resistance around 50–54°C/W, the temperature rise could be approximately:

$$\Delta T=P_{loss}\theta_{JA}=7W(50^{\circ}C/W)=350^{\circ}C$$

That explains why a buck converter is the better choice: it wastes much less power and keeps the board cooler. The LM7805 family is rated for up to about 1–1.5 A depending on conditions, but datasheets emphasize heat sinking and thermal limits. [20]

1.1.1.MOSFET Thermal Management

A heat sink was added to the MOSFET to improve thermal performance during high-current operation. Since the MOSFET is placed in the main power path, it may dissipate heat when motor current flows through it. The MOSFET power loss can be estimated using:

$$P_{MOSFET} = I^2 R_{DS(on)}$$

For example, if the motor system draws 5A and the MOSFET on-resistance is approximately 0.02Ω:

$$P_{MOSFET} = 2W$$

This heat can raise the MOSFET temperature significantly if no cooling is used. Adding a heat sink increases the surface area available for heat dissipation, reducing the MOSFET junction temperature and improving reliability. This was especially important because the motors can draw high startup current, causing short but large current spikes through the protection stage. Therefore, the heat sink was included as a practical thermal protection measure to prevent overheating during continuous or high-load operation.

1.1.2. Stabilization Capacitors

To stabilize the power rails, both electrolytic capacitors and ceramic capacitors were used. The electrolytic capacitors act as bulk storage capacitors. They help smooth slower voltage dips caused by sudden load changes, especially when the motors start or change direction. The ceramic capacitors provide high-frequency decoupling and are placed close to sensitive components to suppress fast switching noise. Together, they reduce ripple on the 5 V rail and improve stability for the ESP32 and sensors.

A simple capacitor sizing estimation used:

$$C=I\Delta t/\Delta V$$

For example, if the circuit needs to support a 0.5A transient for 2ms while allowing only 0.5V of voltage drop:

$$C=0.002F$$

This shows why larger electrolytic capacitors are useful near power inputs and motor supply regions, while smaller ceramic capacitors are better for fast local noise filtering.

This combination creates a wide-band filtering effect, where electrolytic capacitors handle low-frequency disturbances and ceramic capacitors suppress high-frequency noise, resulting in improved power integrity across the system.[21]

Additionally, resistors were incorporated for current limiting (e.g., LED protection), pull-up/pull-down configurations, and signal conditioning, ensuring stable logic levels and protecting components from overcurrent conditions.

1.1.3. Resistors and Signal Conditioning

Resistors were also used throughout the board for current limiting, pull-up/pull-down behavior, and signal protection.[22] For the LED headlights, the current-limiting resistor value can be estimated using:

$$R=(V_{supply}-V_f)/I$$

Because the LED is powered from 5 V, has a forward voltage of about 2 V, and the desired current is 10mA, then the desired value of resistor that should have been used should be around:

$$R= 300\ ohm$$

So a nearby practical value such as 330 Ω is reasonable, and that was the value that was used.

1.1.4.Noise and Grounding

Because the motors draw large and changing current, they can inject noise into the power system. The noise spike caused by wiring and PCB inductance can be estimated by:

$$V=L.(di/dt)$$

This is why the layout uses a ground pour, wide traces, and decoupling capacitors. The goal is to keep high-current motor return paths away from the ESP32 and sensor return paths as much as possible. [23]

1.1.5 Motor Driver Circuit Design

The motor control subsystem utilizes four independent motor driver circuits, each dedicated to driving one DC motor (MTR1–MTR4). This approach allows full individual control of each motor, improving maneuverability and simplifying control logic. The drivers are controlled by digital outputs from the ESP32 DevKitC-32UE, which generates the required logic signals through GPIO pins.

The initial design utilized two motor drivers, as the system was originally implemented as a rear-wheel drive configuration, where only the back wheels were actively driven while the front wheels were passive. This approach simplified the hardware and reduced overall current demand. However, during testing, the system exhibited poor turning performance and limited rotational control, particularly during low-speed maneuvers and in-place rotation.

To improve maneuverability, an attempt was made to connect the front motors to the existing motor drivers, effectively driving two motors per driver. While this reduced hardware complexity, it introduced a critical issue during operation. DC motors draw a high startup (stall) current, and when two motors were connected to a single driver, the combined startup current demand could exceed the driver's current capability.[25]

The startup current can be approximated by:

$$I_{start} = V / R_{winding}$$

So total current per a single motor driver when it was supplying 2 DC motors :

$$I_{total} = 6 + 6 = 12A$$

And the measured current during the trial was around 11A. This current significantly exceeds the typical current rating of DRV8251[24] , explaining the observed issues such as failure to start, voltage drops, and unstable operation. This confirms that the drivers were being overloaded during startup conditions.

To resolve this limitation, the design was modified to include two additional motor drivers, resulting in a total of four independent motor driver circuits, one per wheel. Due to layout and integration constraints on the main PCB, the two additional drivers were implemented on separate breakout boards, with one motor driver per board, and connected externally to the main system.

This updated four-wheel drive (4WD) architecture enabled independent control of all motors, eliminating the overcurrent issue and significantly improving system performance. With each motor driven by its own dedicated driver, the system achieved reliable startup, improved torque distribution, and precise turning control. Although this increased system complexity and total current consumption, it provided a robust and scalable solution that met the system's performance requirements.

Distributed Driver Architecture

Instead of using a single multi-channel driver, the system employs four separate motor driver units, each handling one motor. This design choice provides several advantages:

- Load distribution: Each driver handles only one motor, reducing thermal stress per device
- Improved reliability: Failure in one driver does not affect the others
- Simplified routing: Each driver is placed close to its corresponding motor output, minimizing trace length and losses

Each driver is connected directly to the 12 V power rail, allowing it to supply sufficient voltage and current to the motor.

Current and Performance Considerations

Each DC motor draws both a nominal operating current and a significantly higher startup (stall) current, which can be approximated by:

$$I_{start} = R_{winding} / V$$

Because the winding resistance is typically low, the startup current can be several times higher than the steady-state current. With four motors operating simultaneously, the total current demand becomes:

$$I_{total} = I_1 + I_2 + I_3 + I_4$$

This cumulative current places significant demand on:

- The power supply
- PCB traces
- Protection components (fuse, MOSFET)

This explains the observed heating and voltage drops during testing.

Thermal Distribution Advantage

Using four separate drivers distributes power dissipation across multiple components. By splitting the load across four drivers, each device dissipates less power compared to a single centralized driver. This reduces localized heating and improves thermal management of the system.

Noise and Switching Effects

Each motor driver introduces switching noise due to the inductive nature of the motor load. With four drivers operating, noise sources are effectively multiplied. To mitigate this:

- Decoupling capacitors were placed near each driver
- Motor traces were kept short and wide
- Power and signal routing were separated

This reduces interference with the ESP32 and other sensitive components.

System Integration

Each motor driver is controlled independently via ESP32 GPIO pins, enabling flexible control strategies such as differential drive and directional control. The drivers are connected to the 12 V SAFE rail, ensuring that all motor current is drawn from the high-power domain, while the control signals remain in the low-power domain.

This architecture allows:

- Independent motor control
- Reduced electrical interference
- Improved system modularity

1.2.Remote Control Electrical Architecture (RC PCB)

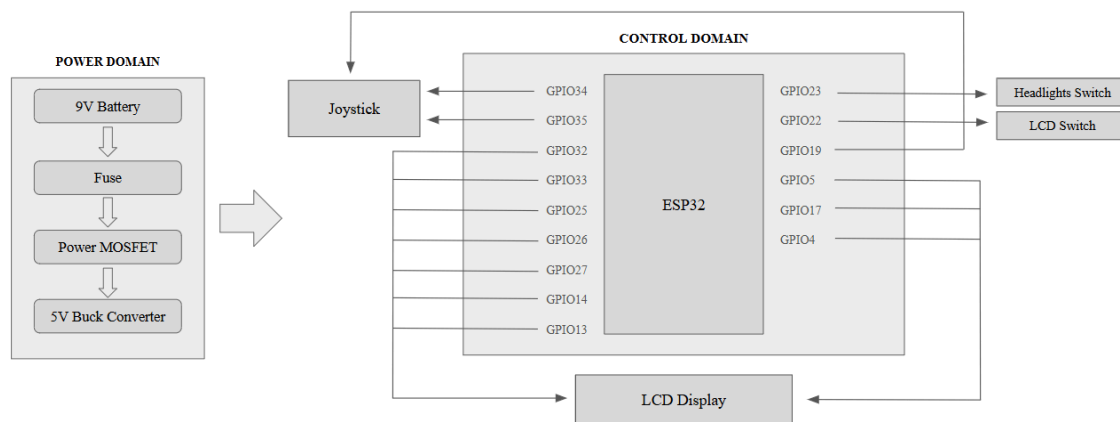


Fig.2. Remote Control Electrical Architecture

The control PCB is responsible for processing user input, generating control signals, and interfacing with peripheral devices. It is centered around the ESP32 DevKitC-32UE, which acts as the main processing unit. The control domain integrates multiple subsystems, including joystick input, LCD display output, motor control signals, and auxiliary switching functions such as headlights and display control.

1.2.1. System Integration and Signal Flow

The control PCB operates within the low-voltage domain (5 V / 3.3 V) and is powered by the regulated 5 V rail from the power board. User input is provided through a joystick module, which generates analog signals read by the ESP32 via ADC-capable GPIO pins (e.g., GPIO34, GPIO35). These inputs are processed to generate motion commands.[26]

The ESP32 outputs digital control signals through GPIO pins to the motor drivers located on the power board and external breakout boards. Additional GPIOs are used to control LEDs and switching functions. The LCD display is interfaced using a parallel communication scheme, requiring multiple GPIO lines for both data and control signals.

1.2.2. PCB Layout and Routing Strategy

The layout of the control PCB was designed to support a high density of signal routing while maintaining signal integrity. The ESP32 is centrally positioned to minimize trace lengths to all peripherals. The LCD interface uses organized parallel routing, with traces kept short and consistent to maintain timing alignment.

Signal routing avoids unnecessary crossings and maintains separation between functional blocks. A continuous ground plane is used to provide a low-impedance return path and reduce electromagnetic interference, which is critical due to the large number of digital signals and proximity to motor control circuitry.

1.2.3. Signal Integrity and Voltage Interface Challenges

A key challenge in the control PCB design was interfacing the 3.3 V logic signals of the ESP32 with the 5 V logic requirements of the LCD display. According to the LCD datasheet, the display requires a 5 V supply and corresponding logic levels, with a logic HIGH threshold approximately given by:

$$V_{IH} \approx 0.7 \times V_{CC} = 3.5V$$

Since the ESP32 outputs a maximum of 3.3 V, this falls slightly below the required threshold, which can lead to unreliable communication. To address this, a voltage level shifting circuit was initially implemented between the ESP32 and the LCD to convert 3.3 V signals to 5 V[27].

During testing, it was observed that the LCD powered on correctly, indicating that the level shifter successfully provided the required voltage. However, no valid data was displayed, suggesting that the data signals were not being properly transmitted.[28] This indicated that the level shifting circuit, while providing the correct voltage level, was introducing issues such as:

- Signal distortion or attenuation
- Insufficient switching speed for parallel data transfer
- Incorrect directionality or timing mismatches

As a result, the level shifter was removed and the LCD was directly connected to the ESP32. Under this configuration, the LCD began to function correctly and display valid data. This demonstrated that

although the signal amplitude was slightly below the datasheet specification, the LCD was still able to interpret the signals reliably, whereas the level shifting circuit was effectively degrading signal integrity.

In addition to voltage interface challenges, electrostatic discharge (ESD) protection was also a critical consideration due to the handheld nature of the remote control. To ensure long-term reliability and protect sensitive components from static buildup, Transient Voltage Suppressor (TVS) diodes were integrated at the input junctions of the joystick and button interfaces. [29] These diodes act as fast-response shunt devices, diverting high-voltage spikes to ground before they can reach and damage the microcontroller. This is particularly important in user-facing systems where repeated physical contact increases the likelihood of electrostatic discharge events.

Another issue encountered during development was related to GPIO configuration. During the initial PCB routing, GPIO2 was used as a general-purpose signal line, as the ESP32 pinout reference used did not explicitly indicate that it functions as a boot-strapping pin.[26] However, during testing, the system exhibited unstable or failed boot behavior depending on the state of this pin at startup. This occurs because GPIO2 influences the ESP32 boot mode and must be held at specific logic levels during reset.

To resolve this issue, the signal originally assigned to GPIO2 was externally rewired to another GPIO pin that does not have boot constraints, restoring reliable system startup. This highlights the importance of carefully considering microcontroller-specific pin functions during PCB design.

Overall, these challenges emphasize that signal integrity depends not only on voltage levels but also on timing, routing, protection mechanisms, and device-specific constraints, all of which must be carefully managed to achieve reliable system operation.

1.2.4.Decoupling and Passive Component Usage

To stabilize the control circuitry, both ceramic capacitors and electrolytic capacitors were used. Ceramic capacitors were placed close to the ESP32 and other integrated circuits to suppress high-frequency noise, while electrolytic capacitors were used for low-frequency filtering and bulk energy storage.[30] Resistors were used for:

- Pull-up and pull-down configurations
- Current limiting (e.g., LEDs)
- Signal conditioning

These passive components ensure stable operation and prevent undefined logic states.[31]

2.Metal Detection

To find a reliable method for our car to detect the copper plates, we went through several design phases. Initially, we focused on designing analog electrical circuits to apply basic electromagnetic principles into our car. After discovering the challenges that came with these circuits, we pivoted to a coding-based approach using the Raspberry Pi Pico to manage signal processing. Ultimately, we transitioned to a contact-based sensing method to ensure our car had the reliability and stability needed for an RC vehicle operating in the final demonstration.

2.1.Phase 1: NE555 Timer Circuit

We began by testing an NE555 timer seen in Figure 3, which was designed to toggle a magnetic field through a copper coil approximately 8cm in diameter. While the initial bench testing was successful, and the buzzer frequency shifted noticeably when the coil was placed near metal, we encountered a major issue when trying to “null” the circuit. We introduced a 50kohm potentiometer to find the edge frequency that would keep the buzzer silent until the metal is detected, but the NE555 didn’t have the precision needed for this. The “silent” threshold was too narrow, and any slight vibration or voltage ripple made it nearly impossible to maintain a reliable zero-state while the car was in motion.

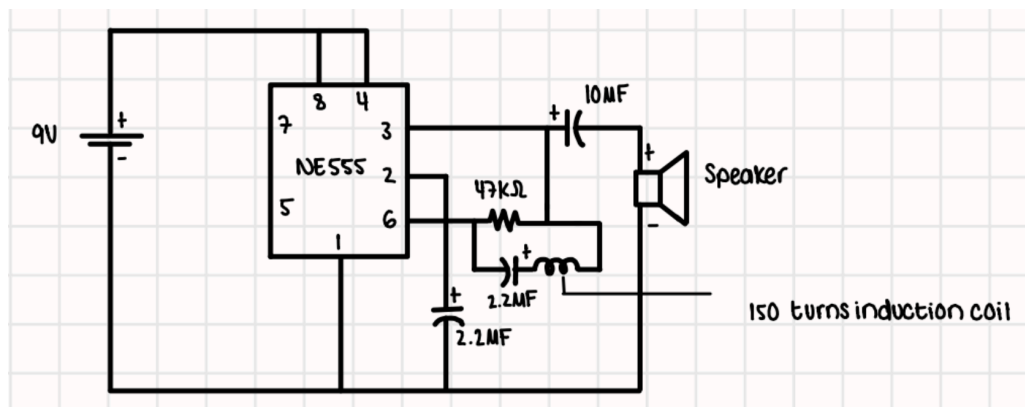


Fig.3. Original NE555 reference circuit before adjusting with potentiometer [17]

2.2.Phase 2: Transistor-Based Oscillator Circuit

To address the sensitivity issues with the timer, we switched to a high-gain oscillator using the BC550 transistors, as shown in Figure 4. This circuit functioned by using the copper coil as a frequency-determining feedback component, where transistors were biased to maintain a steady oscillation powering the buzzer. While some reference designs used lower-gain components to trigger the buzzer only on detection, we only had high-gain transistors on hand, creating a very strong feedback loop. Through testing, we found that it inverted the logic so the buzzer remained on by default because the circuit was pushed so easily into oscillation. Conceptually, the design was based on the principles of induction: as the copper plate entered the coil’s magnetic field, it would generate “eddy currents” that took energy from the circuit. This dampening effect dropped the oscillation amplitude low enough to silence the buzzer upon detection. To correct the inverted logic, we integrated a 50kohm potentiometer to try and “null” the circuit by finding the exact threshold where the oscillation would just barely stop. However, because of the high-gain transistors used, the window was extremely narrow. Due to the lack of

stable tuning range, and because of different materials being used from the reference designs, we moved towards a third iteration before we were able to attach this to the vehicle chassis.

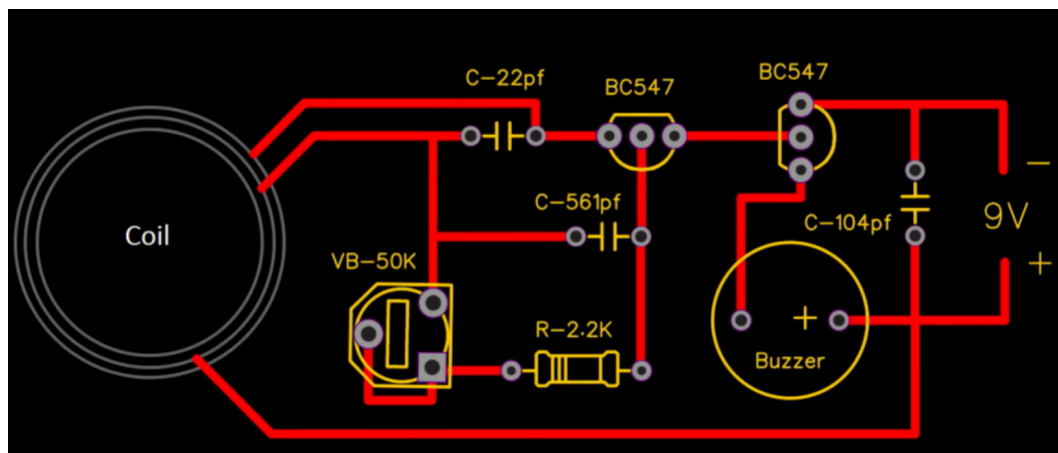


Fig.4. Original transistor-based reference circuit [18]

2.3.Phase 3: Multi-Stage Induction Balance

The final fully electrical attempt was the creation of a multi-stage signal processing chain. This circuit used a 7414 Schmitt Trigger inverter to create a stable 10kHz square wave oscillator, which drove the transmitter (TX) coil through a current-limiting resistor. To process the return signal, a TL072 op-amp was used to amplify any magnetic disturbances picked up by the receiver (RX) coil. This amplified AC signal was then passed through a diode-based envelope detector to rectify it into a DC voltage representing the signal strength. Finally, an LM339N comparator compared this DC voltage against a threshold set by a 10kohm potentiometer.

During bench testing, this approach gave us more control over the detection threshold compared to our previous iterations. However, while the TL072 op-amp provided high-gain amplification, it also made the system extremely susceptible to electrical noise from the DC motors and the car battery. To combat this, we created a custom Faraday cage by shielding the battery with aluminum foil. This was done to prevent any radiated noise from coupling with the inputs of the induction coils. Additionally, despite the effective shielding, we continued to struggle with the mechanical alignment of the TX and RX coils. The induction balance was so sensitive to vibrations that even the slightest shifts in the coils during motion would change the baseline voltage of the comparator, triggering the buzzer. Ultimately, while the Faraday cage and signal chain were very effective, the system was too delicate to be used as our final metal detection circuit.

2.4.Phase 4: Digital Pulse Induction

Taking lessons from the previous iterations, we attempted to utilize a digital pulse induction method controlled by a Raspberry Pi Pico microcontroller. By this time, both our PCB orders had been completed, so we were not able to integrate the metal detection with ESP32, hence why we chose to use a Pi Pico. This design allowed for more precise control over the detection parameters, while simplifying the circuit and taking some of the components from Phase 3. In this system, the Pico would pulse the transmitter (TX) to build a magnetic field and then abruptly cut the pulse to allow the field to collapse. After this, the

Pico used its 16-bit ADC Converter on pin GP26 to listen for the voltage “echo” or decay curve. If metal was present, the induced eddy currents would alter the shape of this echo, which the Pico would then detect as a change in voltage. This method also implemented an automated calibration sequence upon startup. It was programmed to take 100 initial samples to establish a baseline average of the environment’s electromagnetic noise. This baseline allowed the system to set a dynamic threshold to distinguish between metal detection and background fluctuations.

While this digital approach worked well for metal detection regularly, it had one major flaw preventing us from finalizing it. Although the startup calibration accounted for static interference (metal screws, rods, etc.), it could not account for the dynamic electromagnetic interference generated by the motors during operation. Every time the motors spun, they produced unpredictable inductive spikes that the ADC may interpret as valid signal echo. This led to frequent false positives, where the buzzer would be triggered due to motor activity rather than the presence of metal. While the Faraday cage method could be used to remove the electrical noise, the risks of heat building up and damaging our chassis or potential electrical shorts made this impractical.

2.5.Phase 5: Contact-Based Metal Detection

To achieve 100% reliability required for our car, our final iteration pivoted from inductive sensing to a contact-based method. This design removed the many complications we faced of magnetic induction and replaced it with a simple physical circuit that remained unaffected by electromagnetic interference from our DC motors. The hardware used a 9V battery stepped down through a 5V linear regulator to provide a stable power supply to an active buzzer. The logic behind the circuit is that it remains “open” during regular operation; the positive terminal of the active buzzer being connected to the 5V supply, while the negative terminal and the system ground are connected to two separate probes. When the vehicle drives over the copper plate, the plate acts as a conductor, bridging the gap between the probes and completing the electrical path to ground.

The probes are integrated onto the AWD vehicle chassis using a custom 3D-printed attachment shown in Figure 5. The assembly consists of two independent cylindrical mounts fixed to the base of the car, each supporting a hanging, rotatable 3D-printed arm wrapped in the probe wire. These arms were attached with enough rotational freedom to ensure they remain out of the way when moving up the ramp, and are able to snap back into the vertical position once the car returns to leveled ground. By eliminating the challenges we faced in earlier iterations, we were able to find a simple and reliable solution for metal detection in our RC car.

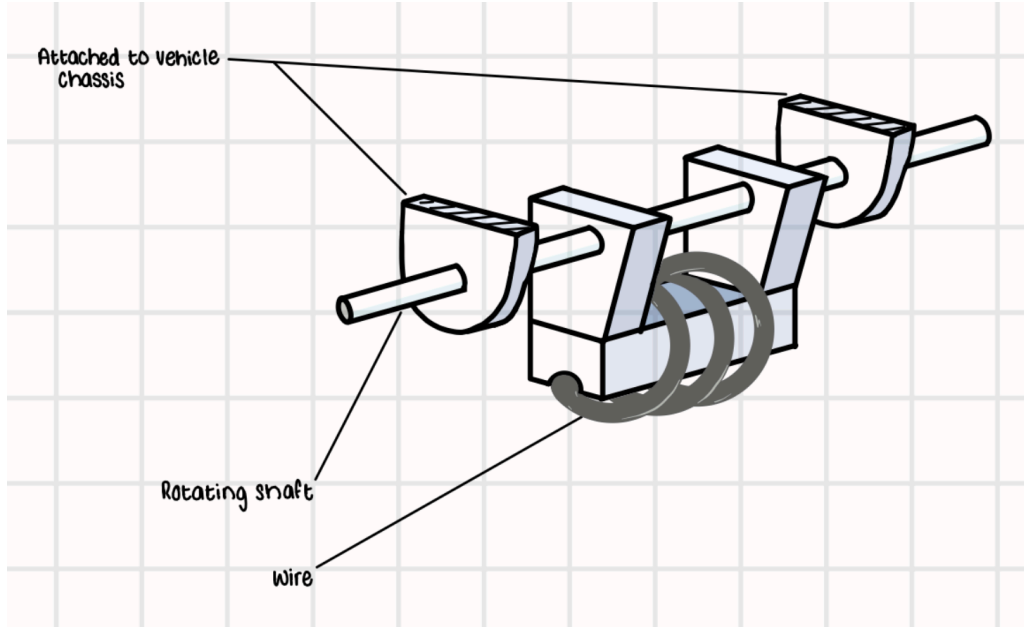


Fig.5. Contact-Based Metal Detection Attachment Drawing

3.Distance Sensing

Ultrasonic Subsystem

To complete the distance sensing portion of our RC car design, we used an HC-SR04 ultrasonic sensor integrated into the front of our vehicle chassis. A small safety concern was the voltage mismatch between the sensor and the ESP32 microcontroller, while the HC-SR04 operates at 5V, the ESP32 GPIO pins are limited to 3.3V.

To protect the microcontroller while ensuring the ultrasonic works well, we added a voltage divider on the sensor's Echo pin, as that is the pin sending information to the ESP32. We selected a 1kohm and 1.5kohm resistor pair. Utilizing voltage division, as shown below, we confirmed that the voltage output was safe for the microcontroller.

$$V_{\text{out}} = V_{\text{in}} * \left(\frac{R2}{R1 + R2} \right) = 5V * \left(\frac{1500}{1000 + 1500} \right) = 3.0V$$

During bench testing of the integrated PCB, we initially encountered an issue where the distance readings were not being sent to our model remote control. While troubleshooting, we discovered inconsistencies with the resistor values placed in the voltage divider circuit of our PCB schematic. Upon discovery, we were able to desolder and replace the resistors with the correct values, and signal stability was restored, and the sensor provided accurate measurements.

4.PCB Routing Strategy and Layout Considerations

The routing and layout of both the power PCB and the control (RC) PCB were designed with a focus on maintaining signal integrity, minimizing power losses, and ensuring reliable operation under high-current and high-noise conditions. Although the two boards serve different functions-power distribution versus signal processing-they were designed using consistent layout principles to ensure seamless system integration.

4.1.Power PCB Routing (High-Current Design)

The power PCB carries significant current, particularly along the path from the 12 V battery input → fuse → MOSFET → motor drivers. To accommodate this, wide copper traces were used to reduce resistive losses and prevent overheating. The resistance of these traces is given by:

$$R=\rho(L/A)$$

and the corresponding power loss:

$$P_{\text{MOSFET}} = I^2 R_{\text{DS(on)}}$$

To minimize these losses, trace lengths were kept short and widths were increased in high-current regions.[32] Routing was designed to be as direct as possible, avoiding unnecessary bends and ensuring efficient current flow. High-current loops were minimized to reduce electromagnetic interference (EMI).[34]

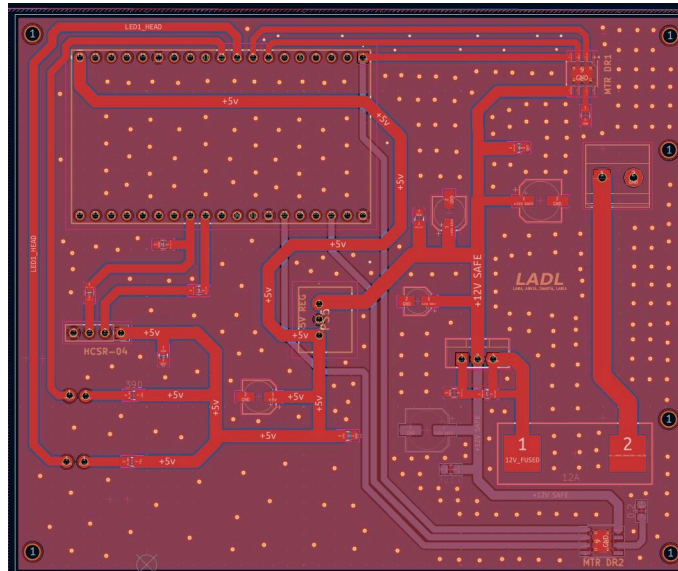


Fig.6. Power PCB Routing Layout

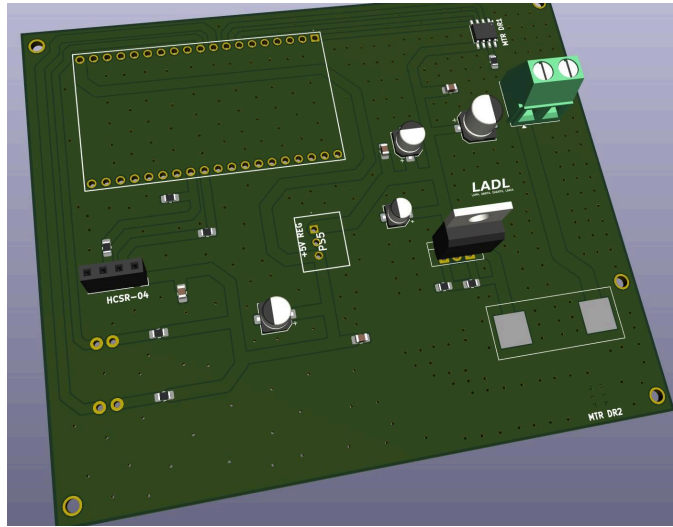


Fig.7. Power PCB 3D Model

4.2.Control PCB Routing (Signal Integrity Design)

The control PCB required dense routing due to the large number of GPIO connections from the ESP32, particularly for the LCD interface operating in parallel mode. Signal traces were carefully organized to:

- Maintain consistent spacing
- Avoid crossings where possible
- Keep trace lengths short to reduce delay and noise

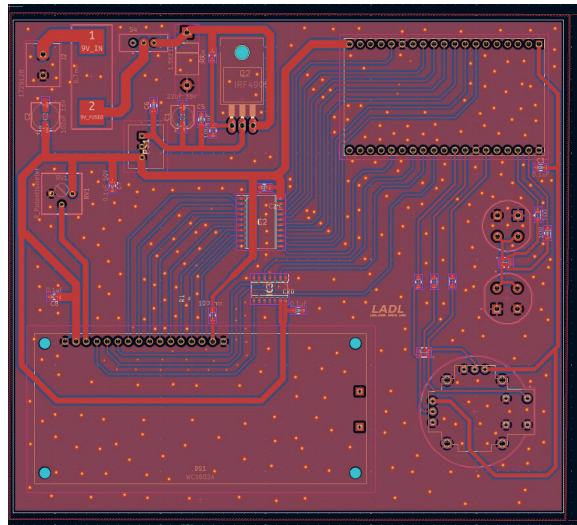


Fig.7. Remote Control PCB Routing Layout

Sensitive signals, such as sensor inputs and control lines, were routed away from high-current paths to prevent coupling of switching noise. The ESP32 was centrally placed to minimize routing complexity and ensure balanced signal distribution.

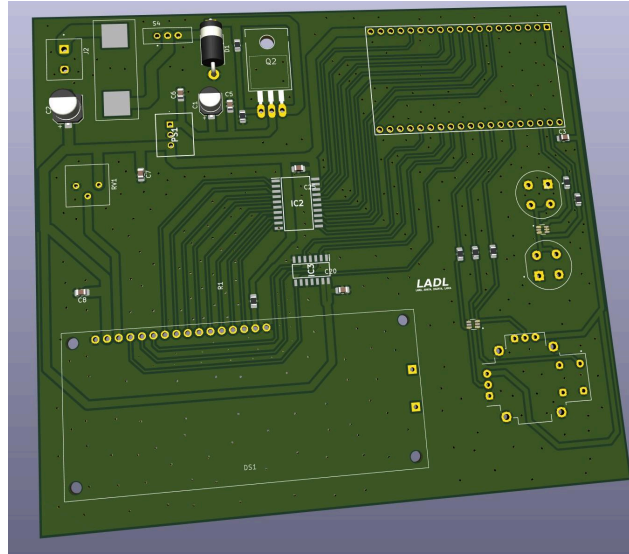


Fig.8. Remote Control 3D Model

4.3. Grounding and Return Path Design

Both PCBs utilized a continuous ground plane to provide a low-impedance return path for current.[35]
This is critical for:

- Reducing voltage drops
- Minimizing ground bounce
- Improving overall noise immunity

Special care was taken to ensure that high-current return paths from motors did not overlap with sensitive logic return paths, reducing interference with the control circuitry.

4.4. Decoupling and Placement Strategy

Proper placement of passive components was essential to routing effectiveness. Ceramic capacitors were placed close to integrated circuits to suppress high-frequency noise, while electrolytic capacitors were placed near the power input and regulator to stabilize low-frequency voltage variations. Their placement minimized loop area and improved the effectiveness of decoupling.[30]

4.5.Routing Challenges and Practical Modifications

During implementation, several routing-related challenges were encountered, including:

- Incorrect GPIO assignment (e.g., GPIO2 as a boot pin)
- Signal degradation due to level shifting circuits
- Voltage drops caused by insufficient trace sizing
- Noise coupling between power and signal domains

Rather than redesigning and manufacturing a completely new PCB, these issues were resolved through practical modifications, including:

- External rewiring of critical signals
- Removal or bypassing of problematic circuit elements
- Use of breakout boards for additional motor drivers
- Minor adjustments to power distribution and grounding

4.6.Design Iteration Without Full Redesign

A key aspect of this project was that all major PCB-related issues were resolved without requiring a full PCB redesign. Instead, a combination of external wiring, modular expansion (breakout boards), and targeted modifications allowed the system to be corrected and improved. This approach significantly reduced development time and cost while still achieving a fully functional and stable system.

This demonstrates an important engineering principle: effective debugging and adaptability can often compensate for initial design limitations, avoiding the need for complete redesign cycles.

5. Remote Controller System Design

The remote controller was implemented using a custom-designed PCB enclosed within a 3D-printed housing with ergonomic handles for ease of use. The layout was optimized primarily for right-handed operation, with the joystick and push buttons positioned on the lower-right side to allow comfortable access using the user's thumb. The controller enables full interaction with the robotic system, including vehicle motion control, headlight operation, and real-time distance monitoring.

5.1. Inputs and Outputs

The control system integrates both analog and digital inputs to provide precise and flexible control of the vehicle.

- **Dual-Axis Joystick:**
The joystick is connected to analog input pins (GPIO34 and GPIO35) of the ESP32 DevKitC-32UE. It operates using internal potentiometers configured as voltage dividers along the X and Y axes. As the joystick is moved, the resistance changes, producing varying voltage levels that are read by the ESP32's ADC. These values are then processed to control the vehicle's speed and steering.[36][26]
- **Push Buttons:**
Two digital input buttons are used to toggle system features such as the LCD display and vehicle headlights. These inputs provide discrete control signals to the ESP32.
- **LCD Display:**
The LCD serves as the primary output interface, displaying real-time distance data obtained from the vehicle's ultrasonic sensor. This allows the user to monitor the environment and make informed control decisions.[27]

5.2. Remote Control Code Design

The code for the embedded system was developed using ESP-IDF in Visual Studio Code for an ESP32 microcontroller for both the car and the controller. The design implements a modular structure that dictates a software module for each hardware component ensuring an efficient system that is simple to debug or alter. The controller is responsible for getting user inputs, transmitting those commands all while the car processes them and returns its own data.

Joystick inputs are handled by using the ESP32's Analog-to-Digital-Converter(ADC) in one-shot mode. This means that the analog voltages from the x and y axes read individually on separate channels/pins when periodically called upon and converted to integer representation from 0 to 4095 that can be used as directional inputs. These values are then transmitted to the car using ESP-NOW, a feature one-shot mode coordinates well with regarding wifi.

The pushbuttons are configured using a pull-up resistor and edge detection logic to ensure the changes in state only occur when the buttons are pressed, not held and have no false readings from toggling. The edge detection is done by comparing the current state with the previous state, only causing a change when a transition from high to low is detected.

ESP-NOW wireless communication between the car and control operates in station mode so the microcontrollers simply use MAC addresses to send small packets of information through a fixed common wifi channel. During initialization the protocol would register callbacks and the peer device containing the other ESP's MAC address was set in place. Different types of data were sent as raw bytes such as floats, ints, and an array so they could be differentiated once received and placed into variables accordingly. Incoming information from the car like ultrasonic distance measurements were handled in a receive callback function. The raw bytes were turned into a usable format, specifically floating-point variable by memory copying. The method was preferred to bluetooth modules or other correspondence methods between two ESP32 devices because it provided easy implementation, so no network router or pairing needed, operated on low power, had a larger range, and was generally more accurate and efficient.

The LCD module works in an 8-bit parallel mode that continuously updates the display to show the most recent data received. The code sends instructions by toggling the enable pin and sending characters via GPIO's that act as data lines. It can also be turned on or off using a pushbutton for power efficiency, this works by simply clearing the display to avoid printing any data.

6.Car Code Design

The motor control operates by using two GPIO outputs per motor driver. Depending on the x and y data received, if the value is above or below a calibrated centre reading the car will move forward or right and backward or left. If the only reading is the centre value the car will simply stop. The actual motor command is made by setting the pins to high or low depending on the direction.

Distance measurements obtained from the ultrasonic sensor are designed to send out a trigger pulse to initiate the process, the echo pulse makes a round trip from and to the sensor in time. Two time readings are done to identify when the echo is sent and retrieved, while statements that wait till this step has been accomplished protect the system from timing conflicts. The difference in time is converted to distance to be sent back to the controller.

The wireless communication for the car works the same way as the ESP-NOW protocol found on the controller. The only difference is that the data being received has very distinct formats, one is an array and the other is an integer, this is done so the data can be assigned to certain variables without mixing up the raw data. If the packet size is two integers that data is stored using memory copy as the joystick information and the remaining packet of one byte is assumed to be the led state.

7.Software Architecture

The software architecture is set around a main loop that calls on three primary modules including input processing, output control and communication. Specifically, the input modules read joystick values as well as button states. This also includes the HC-SR04 sensor readings on the car. The ESP-NOW handles the communication of this information by continuous transmitting and receiving. Finally, on the output

side the display is located on the controller and the headlights and motor functions can be found on the car. The overall structure was kept largely intact for the semester, except for some simplifications in the metal detection circuit that eradicated the need for a software element.

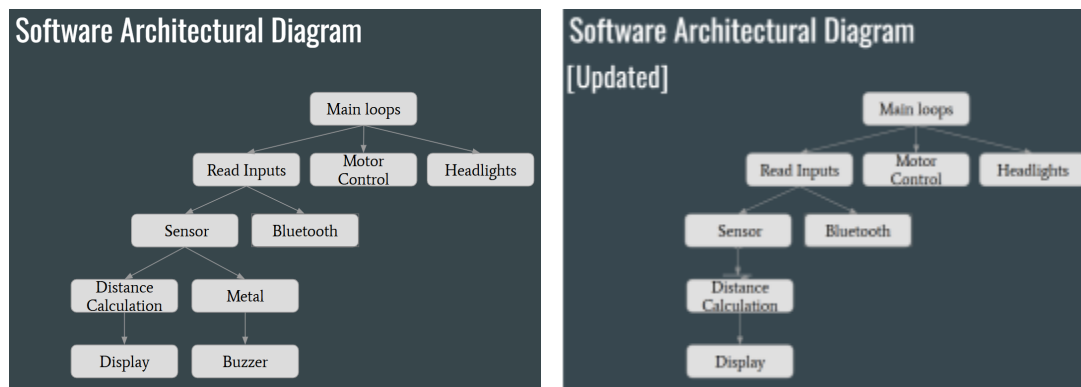


Fig.9. Software Architecture Diagrams

8. Software Design

The system uses two ESP32 microcontrollers, one for the car and the other for the controller. This replaces the STM32 board that had been initially considered. The decision to replace it was made due to the significant simplifications the ESP32 boards brought to the overall design. This included the integrated wifi it processed with its ESP-NOW communication protocol that eliminated the need for additional hardware. The efficiency of the singular IDE and its adjustable pin configuration made the switch an ideal choice. The said peripherals for the two systems can be seen in *table 1* and *table 2*.

There are four key software modules, joystick, esp-now, lcd, motor control, and ultrasonic. The first reads analog values using `adc_oneshot_read()` and converts them into readings that can be used for direction. The second handles data transfer for two directions through the use of callbacks, peer registration and initialization. The lcd module simply displays sensor readings from the car and the motor control module takes potentiometer readings to determine the direction for the car to move. The final module measures distance using an ultrasonic sensor and sends those results.

Timing was dealt with through a continuous loop that had delays embedded in it to ensure no watchdog timeouts were triggered. A clear example of real time behavior prevention can be seen in the several while loops in the ultrasonic module. Another possible timing issue was the esp now callbacks, however those handle asynchronous communication so data was immediately processed separately as not to interfere with the main loop.

Throughout the development of this program several debugging strategies were used and permanently implemented to guarantee continued success and eliminate any technical/hardware issues that may arise. One method includes the testing of simpler components such as lcd, motors, and ultrasonic that could be wired on a breadboard, independently before integrating together. Other strategies are using serial prints along with saving sprint and testing iterations for comparison and isolating issues.

9. Mechanical Architecture

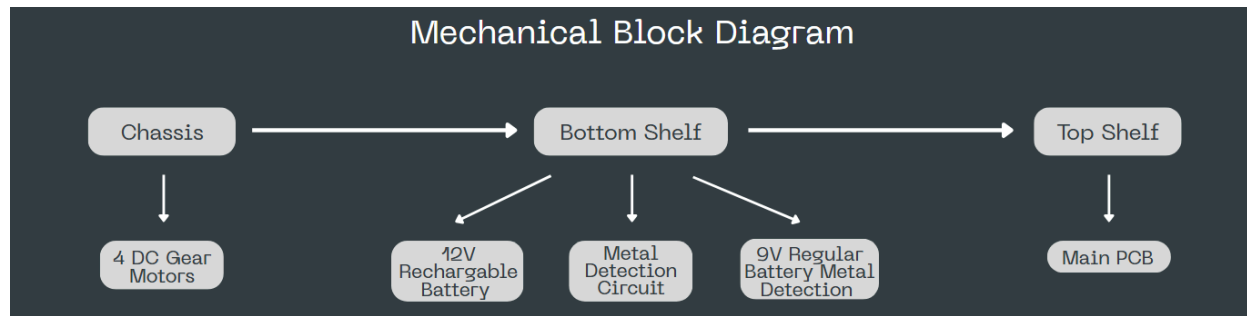


Fig.10. Mechanical Architecture Diagram

9.1. Mechanical Design

From the beginning of our design process, one of our main mechanical goals was to achieve high ground clearance. This was important because the car needed to drive up a ramp with an incline of about 20 degrees, as described in the project requirements. We realized that if the clearance was too low, the chassis could hit the surface of the ramp, which would increase friction and might stop the wheels from rotating properly. By raising the chassis higher off the ground, we reduced the chances of the body interfering with the ramp and made the car's movement smoother and more reliable.

When deciding on the outer shape of the vehicle, we considered two main options: a truck or a Jeep enclosure. A truck design would have given us an open bed at the back, but we chose a Jeep-style design because it provides a much larger enclosed space inside the vehicle. This choice was mainly based on the number and size of components we needed to fit into the car.

The internal system included a custom PCB (approximately 12" × 13"), a perfboard with a regular 9V battery for the metal detection circuit (which replaced an earlier plan to use a larger breadboard), a rechargeable 12V battery, and all the wiring needed to connect these components. Because of this, it was important for us to maximize the available internal space while still keeping the overall size of the car reasonable. The Jeep design allowed us to balance having enough internal space with keeping the car compact, making it the best option for our needs.

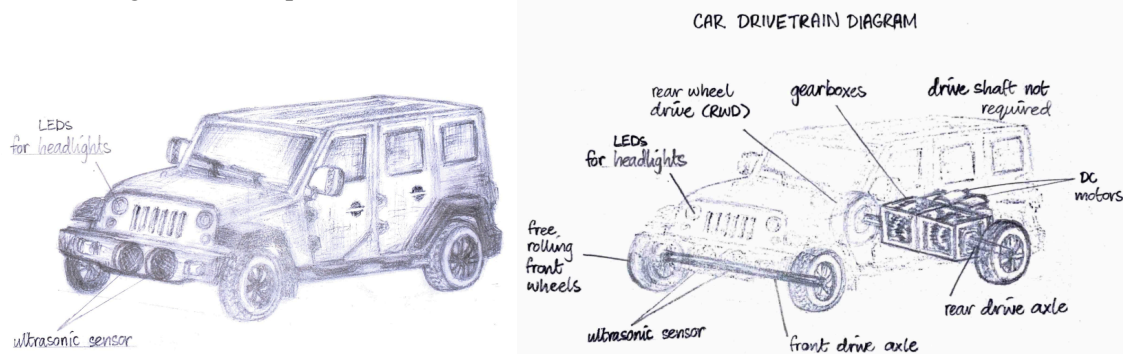


Fig.11. Outer Cover and Initial Drivetrain Diagram Sketches

9.2. Drivetrain Design and Rationale

The drivetrain for our car (see Figure 11) went through several major changes during the project, with each version being improved based on testing and the problems we observed.

At the start, we used a rear-wheel-drive (RWD) setup, where two DC motors were connected to gearboxes that powered the rear wheels, while the front wheels were free-rolling. We chose this design based on basic vehicle dynamics and the initial research we conducted. When a car goes up a ramp, more of its weight shifts to the back, which increases the force on the rear wheels. This gives the rear wheels better grip, making RWD a good choice for climbing inclines (see Figure 12).

However, during testing, we realized that the motors and gearboxes were not providing enough torque to move the car, especially when going up the ramp. Even though the wheels spun very fast when the car was lifted in the air, they stopped moving once the car was placed on the ground. This showed that while the motors had high speed with no load, they did not have enough torque to handle the weight of the car under real conditions.

To solve this issue, we focused on improving the gearbox design, which ended up being one of the most challenging and iterative parts of the project. In our first attempt, we created a fully 3D printed gearbox. While the design itself was fairly complex, it did not provide a high enough gear ratio, so the torque output was still too low to effectively drive the car.

For the second iteration, we redesigned the gearbox using plastic gears along with 3D printed axles. This helped reduce weight, but it introduced a new problem. Under load, the plastic gears wore down very quickly. After only a few test runs, the gear teeth were noticeably damaged and looked “shaved,” which showed that the material could not handle the forces in our system. On top of that, the gearbox produced a lot of noise, which was a concern because it may have interfered with the ability to hear the metal detection buzzer.

In the third iteration, we switched to metal gears to improve strength and durability. While this solved the issue of gear wear, it made the gearbox much heavier. This added weight negatively affected the overall power-to-weight ratio of the car, making it harder for the motors to drive the system effectively. As a result, performance did not improve as expected, and the gearbox was still quite noisy.

Across these first three iterations (see Figure 13), we were using PAN14 DC motors. Based on their specifications, these motors are relatively small and do not produce a large amount of torque. While they are suitable for lighter applications, they were not strong enough for our car, especially when combined with inefficient or heavy gearbox designs.

At one point, we also considered switching to an all-wheel-drive (AWD) system using four motors and two gearboxes (one at the front and one at the back) to increase the total torque and improve traction. However, this would have added even more complexity and weight to our drivetrain due to the additional gearboxes, so we ultimately revised our approach.

After multiple unsuccessful gearbox attempts, we made a major change to our design. Instead of continuing to improve the external gearbox, and purchasing more of the PAN 14 motors, we switched to AWD using 4 DC motors with built-in gearboxes (see Figure 12). This became our fourth and final iteration. By using these motors, we were able to completely eliminate the need for external gearboxes.

Fig.13. Gearbox Iterations: 3D Printed (left-most) vs. Plastic Gears (middle) vs. Metal Gears (right-most)

9.3.Chassis Design and Rationale

The chassis was the main structural base of the vehicle, and it changed quite a bit as our drivetrain design evolved throughout the project.

Our initial design was a simple rectangular base with square cutouts at the rear to fit the gearboxes. We also included a frame in the center at the bottom of the chassis to attach the copper coil we used in our first metal detection method (in the NE55 Timer Circuit). In addition to this, we added a hole through the side of the base for the front wheel free-rolling axle to go, as well as small holes throughout the chassis for wires to pass through.

As our gearbox designs changed, we updated the chassis to match. The square cutouts were changed to rectangular ones so they would better fit the shape of the gearboxes and provide more stable mounting. Later on, when we transitioned to an all-wheel-drive (AWD) setup, we redesigned the chassis again to include four rectangular mounting cutouts for the motors, with two at the front and two at the back.

At the same time, our metal detection method also changed from an inductive approach to a contact-based method. Because of this, the central frame that was originally used to hold the copper coil was no longer needed, so we removed it, therefore, making the chassis layout even simpler.

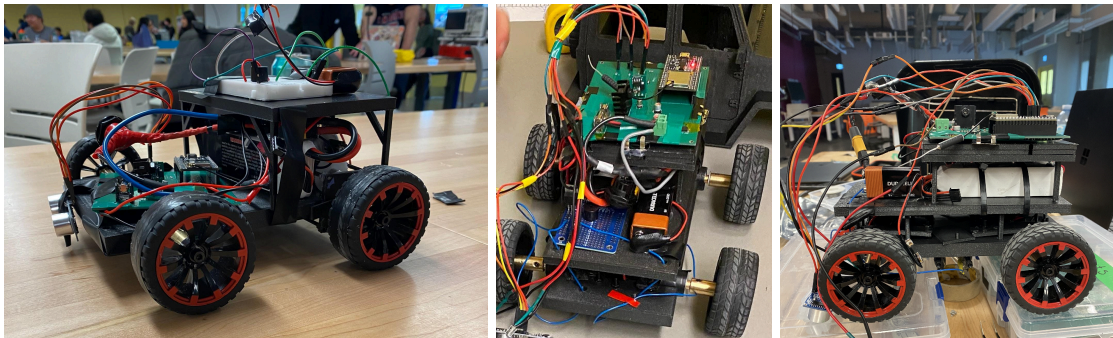


Fig.14. Initial Singular Shelf with Diagonal Braces VS Final Multi-Level Shelves

9.4.Shelves and Mounting Strategy

To better fit all of our electrical components, we introduced a multi-level shelving system, as shown in Figure 14. Instead of spreading everything out across a larger base, the shelves allowed us to use a vertical organization method. This enabled us to keep the chassis compact, keep the components separated, and make better use of the available space inside the car.

At first, we added shelves mainly to prevent electrical interference between different components, especially between the battery and the more sensitive components like the metal detection circuit and PCB. Later on, just to be safe, we also created a Faraday cage around the battery, by wrapping it in aluminum foil, to reduce electromagnetic noise. Although we added the Faraday cage later on, the shelves were still very useful for keeping everything organized and structurally stable.

In the final design, we used:

- A bottom shelf that had the same dimensions as the chassis
- A top shelf that was shorter in length so it wouldn't interfere with the outer cover of the car

The overall outline of both shelves together was designed to match the same shape as the Jeep cover, so everything would fit inside neatly.

We carefully designed the mounting system for the shelves to make it both stable and easy to assemble. The legs of the shelves had a square cross-sectional shape, with small cylindrical pegs extending downward. These pegs fit tightly into matching holes in the chassis. The square shape of the legs acted as a stopper, preventing the shelf from sliding all the way through the holes, while the cylindrical pegs ensured a tight and secure fit.

In earlier versions, when the shelves were taller, we added diagonal braces to the legs to prevent them from bending or breaking under the weight of the components. However, once we reduced the height of the shelves in the final design, these braces were no longer needed.

At the back of the bottom shelf, in the space between the bottom and top shelves, we secured the main 12V rechargeable battery using zip ties. At the front of the bottom shelf, we screwed in the perfboard and secured the smaller 9V battery next to it using double-sided gel tape. On the top shelf, we mounted the PCB by screwing it into place to keep it secure.

For the motors, we used 3D printed brackets to hold them in place. These brackets were screwed into the chassis from both the top and the bottom, which helped keep the motors firmly secured and reduced vibrations during operation. To connect the motor shafts to the wheels, we used metal couplers and screws, ensuring that the wheels were securely attached to the motors and did not roll off during operation.

Lastly, the car cover already fit tightly over the car with all the wiring and components inside, so we did not need to permanently attach it. Instead, we simply placed it on top. We also wanted the cover to be easy to remove so that we could quickly access the internal components when needed, such as for recharging the battery or showing the internal setup during the final demo.

9.5.Materials Selection

All of our structural components were 3D printed using PLA filament. We chose to use 3D printing because of several advantages it offered:

- High design flexibility, allowing us to print any shape or design we needed without major limitations
- Fast prototyping and easy iteration, which helped us quickly test and improve our designs
- No material cost, helping us stay within the project budget
- Sufficient strength for our design needs, making it a practical choice for our structural components

3D printing also allowed us to quickly make changes to our designs based on testing results, which was critical given how many iterations our project went through.

9.6.Mechanical Constraints and Challenges

Several key challenges influenced our design:

- Weight limitations: Too much weight reduced how effectively the drivetrain could move the car
- Torque limitations: The motors struggled to provide enough torque when the car was under load
- Gear wear and failure: Plastic gears wore down quickly and could not handle repeated use
- Space constraints: The large PCB and batteries required proper utilization of the available space inside the car in order to be able to fit all of the components
- Mechanical noise: The car's gearbox being too loud could overpower the buzzer, making it difficult to hear the beep when the car would detect the copper plates
- Noise interference: Electromagnetic noise from the battery had the potential to affect the PCB and metal detection circuit

9.7.Testing and Validation

We tested the mechanical performance of our car through multiple rounds of testing. These tests helped us understand how the car behaved under different conditions and guided the changes we made throughout the design process. Some of the main tests we performed included:

- Running the car while it was lifted in the air and then on the ground to compare how it performed with and without load
- Testing how well the car could climb the ramp
- Observing gear wear over time to check durability
- Evaluating how much noise the system produced during operation
- Checking how stable the components were and how well they were mounted

The results from these tests directly influenced each design change we made, allowing us to improve the performance and reliability of the car step by step.

9.8.Limitations and Future Improvements

Even though our final design worked well, there are still some areas that could be improved to enhance the overall performance of the car in future versions:

- Increase vehicle speed: Our current motors provide more than enough torque, so future improvements would focus on increasing speed. This could be done by using motors with a higher RPM, or using slightly larger wheels to cover more distance per rotation
- Improve wire management: Better organization of wires could reduce clutter inside the car and make assembly and maintenance easier
- Secure the outer cover: Instead of relying only on a tight fit, adding clips or simple fasteners could keep the cover more stable while still allowing easy removal
- Improve component layout: Rearranging internal components could help improve weight distribution and overall balance of the car, since currently we have most of our weight at the rear of the car

10. References

- [1] How To Electronics, “ESP32 LCD display interface tutorial,” YouTube, [Online]. Available: <https://www.youtube.com/watch?v=mts782kyWHM>.
- [2] Newhaven Display International, NHD-0116AZ-FL-YBW LCD module datasheet, [Online]. Available: <https://drive.google.com/file/d/1CrB0qJt7LpF0kpmY0Z8nLET2shyFwUPW/view?usp=sharing>
- [3] ESPBoards, “HC-SR04 ultrasonic sensor with ESP-IDF,” [Online]. Available: <https://www.espboards.dev/sensors/hc-sr04/#esp-idf>.
- [4] ESP32 Tutorials, “HC-SR04 ultrasonic sensor with ESP32 using ESP-IDF,” [Online]. Available: <https://esp32tutorials.com/hc-sr04-ultrasonic-sensor-esp32-esp-idf/>.
- [5] Espressif Systems, “ESP-NOW API reference,” [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html.
- [6] Espressif Systems, “ESP-IDF ESP-NOW example code,” GitHub, [Online]. Available: https://github.com/espressif/esp-idf/blob/v6.0/examples/wifi/espnw/main/espnw_example_main.c.
- [7] theom4, “ESP-NOW TX/RX example (RX.c),” GitHub, [Online]. Available: https://github.com/theom4/ESP-NOW_TX_RX/blob/main/RX.c.
- [8] Espressif Systems, “ESP-NOW examples (component registry),” [Online]. Available: <https://components.espressif.com/components/espressif/esp-now/versions/2.5.3/examples?language=en>.
- [9] University of Calgary, “ENEL 300 Lab 1 Part F: LED Slinky,” [Online]. Available: <https://d2l.ucalgary.ca/d2l/le/content/720116/viewContent/7302236/View>.
- [10] Espressif Systems, “LEDC PWM peripheral documentation,” [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/ledc.html>.
- [11] How To Mechatronics, “DC motor control with driver tutorial,” YouTube, [Online]. Available: <https://www.youtube.com/watch?v=E2raPpB2aJ0>.
- [12] Texas Instruments, DRV8251 4.1-A brushed DC motor driver with integrated current regulation datasheet, [Online]. Available: <https://drive.google.com/file/d/1Wh5SMaBYHGoacs5zciUCc5gGOeRiz09r/view?usp=sharing>
- [13] Espressif Systems, “Joystick component example (ESPP library),” [Online]. Available: <https://components.espressif.com/components/espp/joystick/versions/1.0.36/examples/example?language=en>.
- [14] ESPBoards, “KY-023 joystick module,” [Online]. Available: <https://www.espboards.dev/sensors/ky-023/>.

- [15] Espressif Systems, “ADC one-shot mode API reference,” [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s2/api-reference/peripherals/adc/adc_oneshot.html.
- [16] Espressif Systems, “ADC one-shot example code,” GitHub, [Online]. Available: https://github.com/espressif/esp-idf/blob/master/examples/peripherals/adc/oneshot_read/main/oneshot_read_main.c.
- [17] Ludic Science, “How to make a simple metal detector,” *YouTube*. Oct. 02, 2016. [Online]. Available: <https://www.youtube.com/watch?v=MjVYasjfplo>
- [18] Ezi Circuits, “How to make gold detector or metal detector | Best Metal Detector,” *YouTube*. Aug. 29, 2021. [Online]. Available: <https://www.youtube.com/watch?v=kQC1p4h33Gc>
- [19] Espressif Systems, “ESP32 Datasheet,” 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [20] Espressif Systems, “ESP32 DevKitC User Guide,” 2023. [Online]. Available: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp-dev-kits-en-master-esp32.pdf>
- [21] SparkFun Electronics, “HC-SR04 Ultrasonic Sensor Datasheet,” [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- [22] Handson Technology, “HC-SR04 Ultrasonic Sensor User Manual,” [Online]. Available: <https://www.handsontec.com/dataspecs/HCSR04-Ultrasonic.pdf>
- [23] Infineon Technologies, “IRF4905 P-Channel MOSFET Datasheet,” [Online]. Available: https://www.infineon.com/dgdl/Infineon-IRF4905-DataSheet-v01_01-EN.pdf
- [24] Texas Instruments, “LM340/LM7805 Voltage Regulator Datasheet,” [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm340.pdf>
- [25] Murata Manufacturing, “Decoupling Capacitor Application Guide,” [Online]. Available: <https://www.murata.com/en-us/products/emc/emifil/library/knowhow/basic/chapter06-p1>
- [26] Espressif Systems, “ESP32 Datasheet,” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [27] Hitachi, “HD44780 LCD Controller Datasheet,” [Online]. Available: <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>
- [28] Texas Instruments, “Understanding Logic Level Shifting,” [Online]. Available: <https://www.ti.com/lit/an/scea044/scea044.pdf>

- [29] Littelfuse, “TVS Diode Application Guide,” [Online]. Available: https://www.littelfuse.com/~media/electronics/application_notes/tvs_diode/littelfuse_tvs_diode_application_note.pdf
- [30] Texas Instruments, “Understanding Decoupling Capacitors,” [Online]. Available: <https://www.ti.com/lit/an/slva630/slva630.pdf>
- [31] SparkFun Electronics, “Resistors Tutorial,” [Online]. Available: <https://learn.sparkfun.com/tutorials/resistors/all>
- [32] J. R. Barnes, “Electronic System Design: Interference and Noise Control,” Prentice Hall.
- [33] IPC, “IPC-2221: Generic Standard on Printed Board Design,” IPC Association.
- [34] H. W. Ott, “Electromagnetic Compatibility Engineering,” Wiley, 2009.
- [35] E. Bogatin, “Signal and Power Integrity – Simplified,” 3rd ed., Prentice Hall, 2018.
- [36] SparkFun Electronics, “Voltage Divider Tutorial,” [Online]. Available: <https://learn.sparkfun.com/tutorials/voltage-dividers/all>

Appendices

Bill of Materials

Our team focused on being cost-efficient by using materials we already had on hand. As shown by the blue highlights in Figure 15, many of the more expensive parts, like the ESP32 modules, rechargeable battery, active buzzer, and push buttons, were provided by various team members from their previous projects. This selection of materials was why we decided to pivot to this ESP32 module to begin with. We also sourced general supplies like 9V batteries, wires, and screws for free from Makerspace. The full breakdown of costs can be seen in Figure 15.

RC Car Project					Budget Summary		
Members:	Anaya, Daanya, Lama, Lamia				Budget	Actual	Over/Under
Date:	April 12th, 2026				\$120	\$119.54	Under
ITEM	QUANTITY	PRICE PER UNIT	TOTAL PRICE	LINK TO PURCHASE	ORDERED BY		
PCB Parts							
Motor Driver	x4	3.22	12.88	Motor Driver	Anaya		
MOSFET	x2	4.45	8.9	MOSFET Diode	Anaya & Lama		
Fuse	x2	1.08	1.08	Fuse	Anaya		
Buck	x2	8.08	16.16	buck	Anaya		
Heatsink	x2	0.53	1.06	HEAT SINK MOSFET	Anaya		
JoyStick	x1	8.19	8.19	JoyStick	Anaya		
LCD	x1	10.39	10.39	LCD Display	Anaya		
5V Linear Regulator	x1	0.77	0.77	5V Voltage Regulator	Lama		
Active Buzzer							
9V Batteries							
Wires							
Push Buttons							
Emergency Switch							
ESP32-WROOM-32UE							
Mechanical Components							
DC Motors	x4	10.07	40.28	DC Motor	Lama		
Wheels	x1	11.193	11.193	Wheels	Lama		
Motor Shaft	x1	8.632	8.632	Motor Shaft	Lama		
Stainless Steel Rods							
Screws							
Rechargeable 12V Battery							
			Total Cost:	119.535			

Fig.15. BOM

Peripheral and Pin Interface

*Table 1. Car ESP32 Peripherals
and Pin Configuration*

Peripheral	Pin
Ultrasonic trig	GPIO17
Ultrasonic echo	GPIO16
Motor1 IN1	GPIO18
Motor1 IN2	GPIO19
Motor2 IN1	GPIO22
Motor2 IN2	GPIO21
Motor3 IN1	GPIO32
Motor3 IN2	GPIO33
Motor4 IN1	GPIO27
Motor4 IN2	GPIO14
LED1	GPIO25
LED2	GPIO26

*Table 2. Controller ESP32 Peripherals
and Pin Configuration*

Peripheral	Pin
Joystick VRx (ADC CHANNEL 6)	GPIO34
Joystick VRy (ADC CHANNEL 7)	GPIO35
Joystick SW	GPIO19
LCD D0	GPIO13
LCD D1	GPIO14
LCD D2	GPIO27
LCD D3	GPIO26
LCD D4	GPIO25
LCD D5	GPIO33
LCD D6	GPIO32
LCD D7	GPIO5
LCD RS	GPIO17
LCD EN	GPIO4
LCD WR	GND
LCD Switch	GPIO22
Headlights Switch	GPIO23

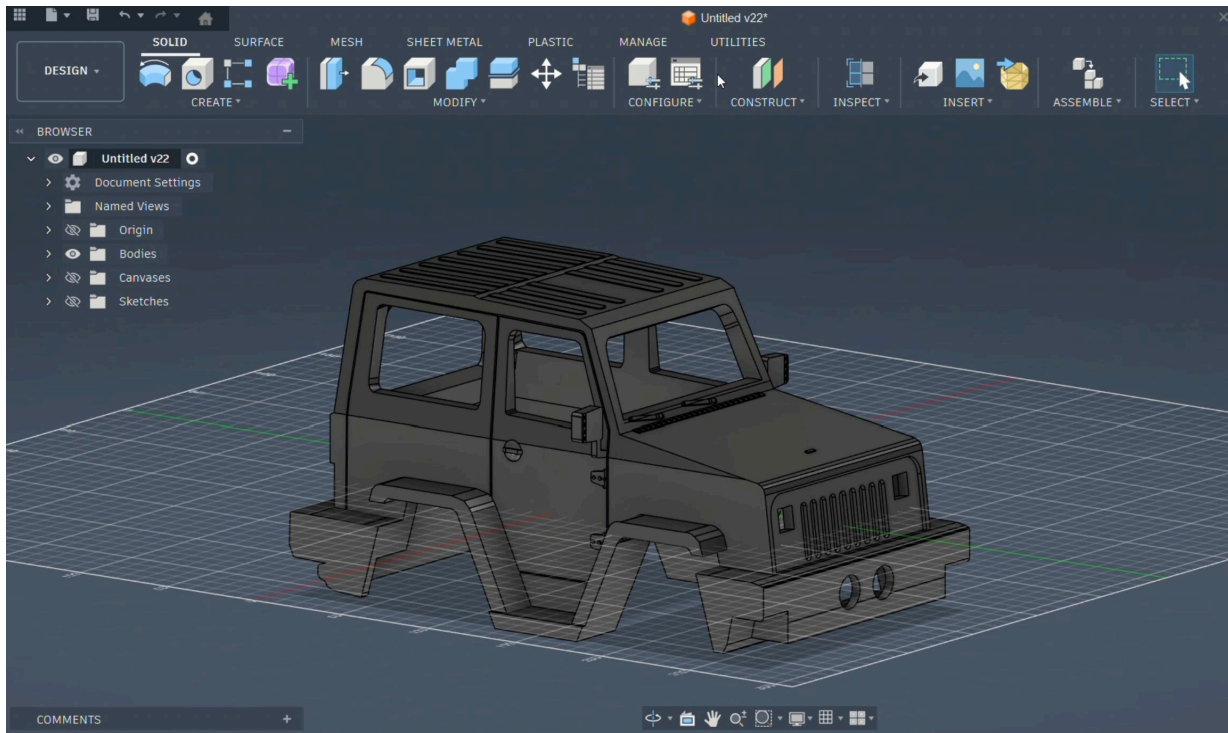


Fig.16. Final Car Cover



Fig.17. Final Car Pictures